Assignment 1: File System Module
Variant: 33968
Student: Mihai Adrian Diaconu

1 Assignment Description

You are given the following binary file format, which we will call from now on **SF** (i.e. "**section file**") format.

A SF file consists in two areas: a **header** and a **body**. The overall SF file structure is illustrated below. It can be noted that the file header is situated at the beginning of the file, before the file's body.

SF FILE STRUCTURE

```
HEADER
BODY
```

A SF file's header contains information that identifies the SF format and also describes the way that file's body should be read. Such information is organized as a sequence of fields, each field stored on a certain number of bytes and having a specific meaning. The header's structure is illustrated in the `HEADER` box below, specifying a name for each header's field and the number of bytes that field is stored on (separated by ": "). Some fields are just simple numbers (i.e. `MAGIC`, `HEADER_SIZE`, `VERSION`, and `NR_OF_SECTIONS`), while others (i.e. `SECTION_HEADERS` and `SECTION_HEADER`) have a more complex structure, containing their own sub-fields. Such, the `SECTION_HEADERS` is actually composed by a sequence of `SECTION_HEADER` fields (elements), each such field in its turn being composed by four sub-fields: `SECT_NAME`, `SECT_TYPE`, `SECT_OFFSET`, and SECT_SIZE.

HEADER

```
MAGIC: 4
HEADER_SIZE: 2
VERSION: 2
NO_OF_SECTIONS: 1
SECTION_HEADERS: NO_OF_SECTIONS * sizeof(SECTION_HEADER)
    SECTION_HEADER: 11 + 2 + 4 + 4
        SECT_NAME: 11
        SECT_TYPE: 2
        SECT_OFFSET: 4
        SECT_SIZE: 4
```

The meaning of each field is the following:

- The `MAGIC` field identifies the SF files. Its value should be `"BTT1"`.
- The `HEADER_SIZE` field indicates the size of the SF files' header.
- The `VERSION` field identifies the version of the SF file format, presuming the SF format could be changed a bit from one version to another, though not the case in your assignment, even if the version numbers could be different between different SF files — see below.
- The `NO_OF_SECTIONS` field specifies the number of the next `SECTION_HEADER` elements, which are covered in the description above by the `SECTION_HEADERS` name (field).
- The `SECTION_HEADER`'s sub-fields are either self explanatory (e.g. `SECT_NAME` or `SECT_TYPE`) or their meaning is explained below.

The SF file's body, basically a sequence of bytes, is organized as a collection of sections. A section consists in a sequence of `SECT_SIZE` consecutive bytes, starting from the byte at offset `SECT_OFFSET` in the SF file. Consecutive sections could not necessarily be placed one near the other. In other words, there could be bytes between two consecutive sections not belonging to any section described in the SF file's header. A SF file's section contains printable characters and special line-ending characters. We would say thus that they are text sections. Bytes between sections could contain any value. They are however of no relevance for anyone interpreting a SF file's contents.

Inside any given section, the line numbering is made from the beginning to the end, starting with 1.

The following restrictions apply to the values certain fields in a SF file could take:

- The `MAGIC`'s value is "BTT1".
- `VERSION`'s value must be between 68 and 155, including that values.
- The `NO_OF_SECTIONS`'s value must be either 2 or a value between 4 and 16, including that values.
- Valid `SECT_TYPE`'s values are: 70 64 77 93 (in base 10).
- Section lines are separated by the following sequence of line-ending bytes (values are hexadecimal): `0A`.

Figure 1 illustrates a SF example, comprising of the following elements:

- `HEADER`: from the offset 0x00 to the offset 0x32 (the beginning of the file)
  - `MAGIC`: at the offset 0x00, having the value `"BTT1"`
  - `HEADER_SIZE`: at the offset 0x04, having the value 51 (0x33)
  - `VERSION`: at the offset 0x06, having the value 71 (0x47)
  - `NO_OF_SECTIONS`: at the offset 0x08, having the value 2
  - `SECTION_HEADERS`: from the offset 0x09 to the offset 0x32:
    - $\texttt{SECTION_HEADER}_1$

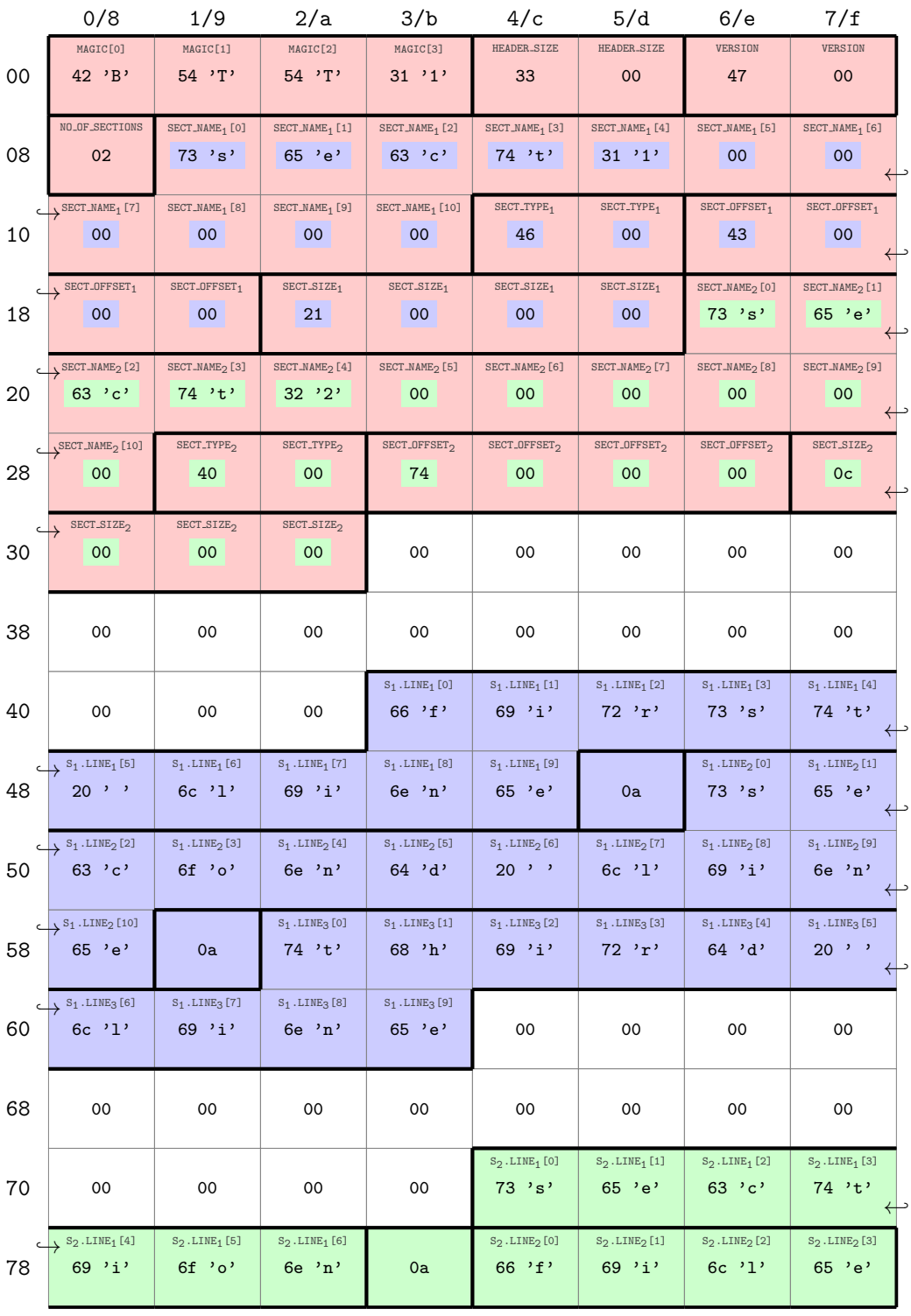


Figure 1: SF file example

        - SECTION_NAME$_1$: at the offset 0x09, having the value "sect1"
        - SECTION_TYPE$_1$: at the offset 0x14, having the value 70 (0x46)
        - SECTION_OFFSET$_1$: at the offset 0x16, having the value 0x43; this means that section 1 starts at the file address 0x43
        - SECTION_SIZE$_1$: at the offset 0x1a, having the value 33 (0x21); this means that section 1 takes up 33 bytes in the file
    * SECTION_HEADER$_2$
        - SECTION_NAME$_2$: at the offset 0x1e, having the value "sect2"
        - SECTION_TYPE$_2$: at the offset 0x29, having the value 64 (0x40)
        - SECTION_OFFSET$_2$: at the offset 0x2b, having the value 0x74; this means that section 2 starts at the file address 0x74
        - SECTION_SIZE$_2$: at the offset 0x2f, having the value 12 (0xc); this means that section 2 takes up 12 bytes in the file

- section 1 : from the offset 0x43 to the offset 0x63
    - line 1 , from the offset 0x43 to the offset 0x4c having the content "first line"
    - line 2 , from the offset 0x4e to the offset 0x58 having the content "second line"
    - line 3 , from the offset 0x5a to the offset 0x63 having the content "third line"
- section 2 : from the offset 0x74 to the offset 0x7f
    - line 1 , from the offset 0x74 to the offset 0x7a having the content "section"
    - line 2 , from the offset 0x7c to the offset 0x7f having the content "file"

2 Assignment's Requirements

Your are required to write a C program named "*a1.c*" that implements the following requirements.

2.1 Compilation and Running

Your C program must be **compiled with no error** in order to be accepted. A sample compilation command should look like the following:

Compilation Command

```
gcc -Wall a1.c -o a1
```

Warnings in the compilation process will trigger a 10% penalty to your final score.

When run, your resulted executable (we name it "*a1*") **must provide the minimum required functionality and expected results**, in order to be accepted. What such minimum means, will be defined below. The following box illustrates the way your program could be run. The options and parameters your program must accept and their meaning will be explained in the following sections.

Running Command

```
./a1 [OPTIONS] [PARAMETERS]
```

2.2 Variant Output

Each student receives a slightly modified variant of the SF format and the assignment's requirements. For the first task, you should be able to output the identifier of your assignment variant, when your program will be run like below.

Display Variant Command

```
./a1 variant
```

Output Sample for Display Variant Command

```
33968
```

2.3 Listing Directory Contents

When being given the "`list`" option, your program must display on the screen the names of some elements (i.e. files, sub-directories) in the directory whose path is specified with the command line "`path`" option in the form illustrated in the following box.

List Directory Command

```
./a1 list [recursive] <filtering_options> path=<dir_path>
```

Note that the command line options could be given in any order, e.g. the "recursive" option could be given before or after the other options.

Which element names must be displayed is determined based on the filtering criteria mentioned below. Each element name must be displayed on a different line, with no blank lines between valid names, in the form illustrated by the box below, i.e. the element's name prepended with the directory path specified by the "path" option. If no name complying the requested criteria is found, nothing but the "SUCCESS" string must be displayed.

Sample Output for List Directory Command (Success Case)

```
SUCCESS
test_root/test_dir/file_name_1
test_root/test_dir/file_name_2
test_root/test_dir/file_name_3
...
```

If the "`recursive`" option is also specified, your program should traverse the entire sub-tree starting from the given directory, entering recursively in all the sub-folders, sub-sub-folders and so on. Found element names must also contain their path relative to the given directory. The two boxes below illustrate the way the program could be run recursively and a possible output for that case, respectively.

Sample List Directory Command

```
./a1 list recursive path=test_root/test_dir/
```

Sample Output for List Directory Command (Success Case)

```
SUCCESS
test_root/test_dir/file_name_1
test_root/test_dir/file_name_2
test_root/test_dir/subdir_1/file_name_1
test_root/test_dir/subdir_1/subdir_1_1/file_name_2
test_root/test_dir/subdir_2/file_name_3
...
```

In case an error is encountered, an error message followed by an explanation must be displayed, in the form illustrated by the following box.

Output for List Directory Command (Error Case)

```
ERROR
invalid directory path
```

The filtering criteria used to select the elements whose names must be displayed on the screen are the following:

- Element's name must start with the sequence of characters specified with the filtering option "`name_starts_with`=string". You should consider any type of elements, e.g. both files and directories.
- The listed element must have "execution" permission for its owner, specified by the filtering option "`has_perm_execute`". You should consider any type of elements, e.g. both files and directories.

Note that even if in practice more filtering criteria could be given in the same command line, our tests use only one single criterion at a moment. Though, your program could consider the more general case, if you want, even if not a compulsory requirement.

Also note that the order of the listed names is not important for our tests, but only their number and value.

2.4 Identifying and Parsing Section Files

When being given the "`parse`" option, your program must check if the file whose path is specified with the command line "`path`" option complies or not the SF format. The way the program could be run for this is illustrated in the following box.

Check SF Format Command

```
./a1 parse path=<file_path>
```

Similar to the note above, the command line options could be given in any order, and your program should support this.

The way the SF format compliance must be checked is based on the following criteria:

- The value of the magic field is the one mentioned before, i.e. "BTT1".
- The values of the file version must be one from the interval mentioned above, i.e. between 68 and 155, including that values.
- The number of sections must be 2 or between 4 and 16, including that values.
- The existing sections' type must be only in the set mentioned above, i.e. 70 64 77 93 .

When all the above criteria are met, the file could be considered as being compliant with the SF format, even if some other inconsistencies could still be found in it, like the size of the file being less than at least one section's beginning (i.e. section's offset) or end (i.e. section's offset plus section's size), or having overlapping sections.

In case the given file does not comply the SF format, an error message must be displayed on the screen, followed by the failure reason mentioning the first field the checking failed on, in the form illustrated by the box below.

Sample Output for an Invalid SF File

```
ERROR
wrong magic|version|sect_nr|sect_types
```

In case the given file complies the SF format, some of its checked fields must be displayed on the screen in the form illustrated by the box below.

Sample Output for a Valid SF File

```
SUCCESS
version=<version_number>
nr_sections=<no_of_sections>
section1: <NAME_1> <TYPE_1> <SIZE_1>
section2: <NAME_2> <TYPE_2> <SIZE_2>
...
```

2.5 Working With Sections

When being given the "`extract`" option, your program must find and display some part of a certain section of a SF file. In particular, a single line must be extracted. The needed command line arguments are given using the "`path`", "`section`" and "`line`" options, for the file path, section number and line number respectively, as illustrated in the following box.

Extract Section Line Command

```
./a1 extract path=<file_path> section=<sect_nr> line=<line_nr>
```

In case an error is encountered, an error message followed by an explanation must be displayed, in the form illustrated by the following box.

Sample Output for Extract Section Line Command (Error Case)

```
ERROR
invalid file|section|line
```

In case the given file is of the SF format and the searched section and line are found, the result should be displayed as illustrated in the box below.

Sample Output for Extract Section Line Command (Success Case)

```
SUCCESS
<line_content>
```

The line numbers are counted from the section start, the first line being line 1.

2.6 Section Filtering

When being given the "`findall`" option, your program must function similar to the case when run with the "`list recursive`" options, though it must search only for SF files that

have at least 2 section(s) of type 93.

The way the program could be run for this is illustrated in the following box.

Find Certain SF Files Command

```
./a1 findall path=<dir_path>
```

In case an error is encountered, an error message followed by an explanation must be displayed, in the form illustrated by the following box.

Output for Find Certain SF Files Command (Error Case)

```
ERROR
invalid directory path
```

Normal output must be displayed in the form illustrated by the box below.

Sample Output for Find Certain SF Files Command (Success Case)

```
SUCCESS
test_root/test_dir/file_name_1
test_root/test_dir/file_name_2
...
```

3 User Manual

3.1 Self Assessment

In order to generate and run tests for your solution you could simply run the "tester.py" script provided with your assignment requirements. The script requires Python 3.x, not Python 2.x.

Sample Command for Tests Generation

```
python3 tester.py
```

Even if the script could be run in MS Windows OS (and other OSes), we highly recommend running it in Linux, while this is how we run it by ourselves, when evaluating your solutions.

When running the script, it generates a "test_root" directory, containing all sort of files and sub-directories used to check your solution. Even if the contents of the "test_root" directory is randomly generated for each student, for the same student it is (with a high probability) the same, independently of how many times you run the script. The only difference could be between different OSes.

After creating the testing directory's contents, the "test_root" script also runs your program against it, displaying the results of the different tests it runs. There should be normally 65 test, though sometimes less than that could also happen. The maximum grade (10) is gained when all tests pass and no penalty is applied (see below). The precise grade is calculated by scaling the number of points your solution gains due to test passing to the 0-10 scale.

Restrictions:

- You are required and restricted to use only the OS system calls, i.e. low-level functions, not higher-level one, in your entire solution, in all lab assignments. For instance, regarding the file accesses, you MUST use system calls like `open()`, `read()`, `write()` etc., but NOT higher-level functions like `fopen()`, `fgets()`, `fscanf()`, `fprintf()` etc. The only accepted exceptions from this requirement are the functions to read from `STDIN` or display to `STDOUT` / `STDERR`, like `scanf()`, `printf()`, `perror()` and functions for string manipulation and conversion like `sscanf()`, `snprintf()`.

Recommendations:

- For string tokenization (i.e. separate a string into elements based on specific separators, like spaces) we recommend you using the `strtok()` or `strtok_r()` functions.

If you whish to see details for the failed tests, you can use the `--verbose` option, that prints both the expected result and your output. Using the option `--test test_name` you can run a single test with the given name, for debugging purposes. For instance, if you wish to run only the test `parse_2` and see the details, you can combine these two options like below:

Sample command for single test with verbose

```
python3 tester.py --test parse_2 --verbose
```

When your assignment is graded, it is run inside a *docker* container. While a correct and deterministic solution will behave in the same way, a bug in your solution may pass unnoticed on your system but may cause a crash / undefined behavior during the "official" evaluation. For this reason, we encourage you to also test your solution using docker before submitting it. To do this, you need to:

- install *docker* on your machine (`sudo apt install docker.io`)
- create a Docker Hub account and login to it from the command line (it is needed for downloading the evaluation image)
- install the *docker* module for Python (`python3 -m pip install docker`)

To test your solution using the *docker* container, you need to provide the `--docker` argument to the testing script.

Sample command for testing with Docker

```
python3 tester.py --docker
```

If you want to keep the docker container after running the tests (for instance, you may want to debug your code inside the container), use the option `--docker-persist`.

3.2 Evaluation and Minimum Requirements

- If your program has compilation warnings, a 10% penalty is applied.
- If your program has memory leaks (found by *valgrind*), a 10% penalty is applied.
- If your program doesn't comply to the required coding style, a penalty up to 10% is applied (decided by your instructor).

Do not cheat as we run plagiarism detection tools on all provided solutions.

Notes: The provided tests are not exactly the tests we will run on your provided solution. Do not write solutions displaying expected results based on testing file names. Besides, check on your code that the required functionality is completed and correctly provided as just running some set of tests does not necessarily cover all possible cases and prove the solution if perfect. It would thus be possible that some exceptional cases to be covered only be tests that we will run.